VitalWatch

See the trend before the threshold.

TRACK
AI for Real-Time Decision Making

FORMAT
Hackathon idea proposal and technical plan

PROTOTYPE
FastAPI · HTML/CSS/JavaScript · ML · Historical hourly replay

A transparent, replayable early-warning research prototype that combines multivariate vital-sign trends, missingness signals, and an interpretable ML model to prioritize clinician assessment.

> *Safety boundary: alerts prioritize review; they do not diagnose sepsis or recommend treatment.*

2. Problem Statement

The core issue

Patient deterioration often develops as a pattern rather than a single dramatic measurement. Heart rate may gradually rise while respiratory rate increases, oxygen saturation declines, blood pressure weakens, and temperature changes. Each individual value may remain below a conventional alert threshold, while the combined trajectory is concerning.

A threshold-only workflow can therefore produce two problems:

- **Late recognition:** an alert appears only after a vital sign crosses a fixed limit.
- **Alert overload:** isolated threshold breaches generate noisy notifications without enough context.

VitalWatch addresses the gap by analyzing recent changes across several measurements and updating a patient-level risk score as new observations become available.

Target audience

The primary users are:

- Clinicians and nursing teams who need to prioritize patient review.
- Clinical informatics and quality-improvement teams exploring early-warning workflows.
- Researchers and datathon participants building transparent deterioration models.
- Health-tech teams evaluating how historical clinical data can support real-time-style decision tools.

The prototype is not intended to make autonomous clinical decisions. It surfaces a ranked worklist and supporting evidence for a qualified human reviewer.

Why now?

Clinical data is increasingly timestamped and available as a sequence of observations, but many demonstrations still focus on static dashboards or isolated threshold rules. There is an opportunity to combine:

- Causal rolling trends rather than future-aware summaries.
- Missingness indicators, because measurement frequency can reflect clinical workflow.
- Patient-level machine-learning evaluation rather than row-level random splits.
- A replayable interface that demonstrates how risk changes over time.

The PhysioNet/CinC Challenge 2019 dataset provides a practical starting point for a narrowly defined sepsis early-warning task. Its records are hourly rather than second-by-second bedside waveforms, so VitalWatch will replay them honestly as historical hourly observations.

3. Proposed Solution Overview

The pitch

VitalWatch is an AI-assisted clinical prioritization dashboard. It receives a historical or synthetic patient record one observation at a time, calculates recent multivariate trends, estimates the risk of a sepsis-related deterioration event within a defined future window, and presents the most concerning patients first with a concise evidence panel.

No IoT device is required for the prototype. A FastAPI replay service releases historical or synthetic rows chronologically so the dashboard behaves like a live system while preserving the data's original hourly resolution.

Key features

- **Patient priority board:** Rank patients by current model risk and risk change.
- **Chronological replay:** Play, pause, step forward, reset, and change playback speed.
- **Multivariate risk scoring:** Combine current values, rolling summaries, trends, and missingness indicators.
- **Threshold comparison:** Show how a simple threshold baseline compares with the learned model.
- **Risk history:** Plot the patient's risk score over time with alert and retrospective onset markers.
- **Vital-sign trends:** Display heart rate, respiratory rate, oxygen saturation, blood pressure, and temperature.
- **Missing-data visibility:** Show missing measurements explicitly rather than treating them as normal values.
- **Alert explanations:** Describe model-associated evidence such as rising respiratory rate or declining oxygen saturation.
- **Simulation labeling:** Clearly mark the interface as historical replay or synthetic demonstration mode.
- **Evaluation view:** Report lead time, sensitivity at a selected false-alert rate, and patient-level performance.

Value proposition

VitalWatch is valuable because it combines a credible ML workflow with a visible, replayable decision interface:

- It detects **patterns and trajectories**, not only isolated threshold breaches.
- It keeps the human decision-maker in the loop.
- It makes the model's current evidence visible rather than presenting an unexplained score.
- It demonstrates real-time-style behavior without requiring sensors, hospital integration, or protected live data.
- It is lightweight enough to build during a hackathon and extensible toward future streaming inputs.

Intended AI contribution

The AI component is a supervised early-warning model. At time `t`, it uses only observations available through `t` and predicts whether the patient will receive a sepsis label within a predefined future horizon, such as the next six hours.

The minimum model is an interpretable logistic regression. A fixed threshold rule serves as the baseline. The dashboard compares their alert timing and false-alert burden.

4. Architecture & Technical Methodology

Tech stack

Layer	Proposed technology	Purpose
Frontend	HTML, CSS, vanilla JavaScript	Custom, aesthetic clinical command-center interface
Charts	Apache ECharts or Chart.js	Responsive multi-series vital and risk charts
Backend	Python, FastAPI, Uvicorn	REST endpoints, replay control, prediction service
Live updates	WebSocket	Push each simulated observation and risk update to the browser
Data processing	Pandas, NumPy	Normalize records and create causal rolling features
ML	scikit-learn	Logistic regression, preprocessing, metrics, serialization
Storage	CSV/JSON for demo; SQLite or DuckDB for persistence	Synthetic data, predictions, replay state, evaluation results
Model artifact	Joblib	Save and load the trained prototype model
Deployment	Local server or container	Reproducible hackathon demonstration

FastAPI is preferred over Django for the MVP because the core workflow is an API, replay engine, and WebSocket stream. Django becomes attractive later if the system requires accounts, role-based permissions, an administrative console, audit logs, or multi-tenant workflows.

Data sources and data strategy

Model development and evaluation

Use the **PhysioNet/CinC Challenge 2019** dataset as the primary real-world starting point. It provides patient-hour records, vital signs, laboratory values, and a `SepsisLabel` outcome. The MVP can focus on:

- Heart rate (`HR`)
- Oxygen saturation (`O2Sat`)
- Temperature (`Temp`)
- Systolic blood pressure (`SBP`)
- Diastolic blood pressure (`DBP`)
- Respiratory rate (`Resp`)

Dashboard replay

Use a small synthetic CSV or JSON file containing stable, worsening, noisy, and missing-data patient trajectories. The synthetic rows are released chronologically by the replay engine. They are for interface demonstration and pipeline testing, not clinical validation.

Required row schema

```
patient_id
timestamp
heart_rate
respiratory_rate
oxygen_saturation
systolic_bp
diastolic_bp
temperature
heart_rate_missing
respiratory_rate_missing
oxygen_saturation_missing
blood_pressure_missing
temperature_missing
sepsis_label                # retrospective evaluation only
```

The model must not receive the future `sepsis_label` during replay. Blank values and explicit missingness indicators should be retained.

ML methodology

Target definition

For a prediction time `t`:

```
Input: all measurements available through time t
Target: whether the first sepsis label occurs within the next six hours
```

The prototype can define sepsis onset operationally as the first timestamp at which `SepsisLabel` equals 1. This is a dataset-based definition and must not be presented as the exact biological onset time.

Causal features

Features are calculated from the current and prior observations only:

- Current vital-sign values.
- Three-, six-, and twelve-hour rolling means, minima, maxima, and variability.
- Linear trend or slope for each vital sign.
- Change from a recent patient-specific baseline calculated using past data only.
- Hours since the last available measurement.
- Count of missing vital signs in the current window.
- Persistence of increasing or decreasing trends.

Models

1. **Threshold baseline:** A transparent rule that alerts when multiple vital signs cross prototype thresholds.
2. **Primary ML model:** Logistic regression with imputation, scaling, rolling features, and missingness indicators.
3. **Optional comparison:** A gradient-boosted tree model if time permits.

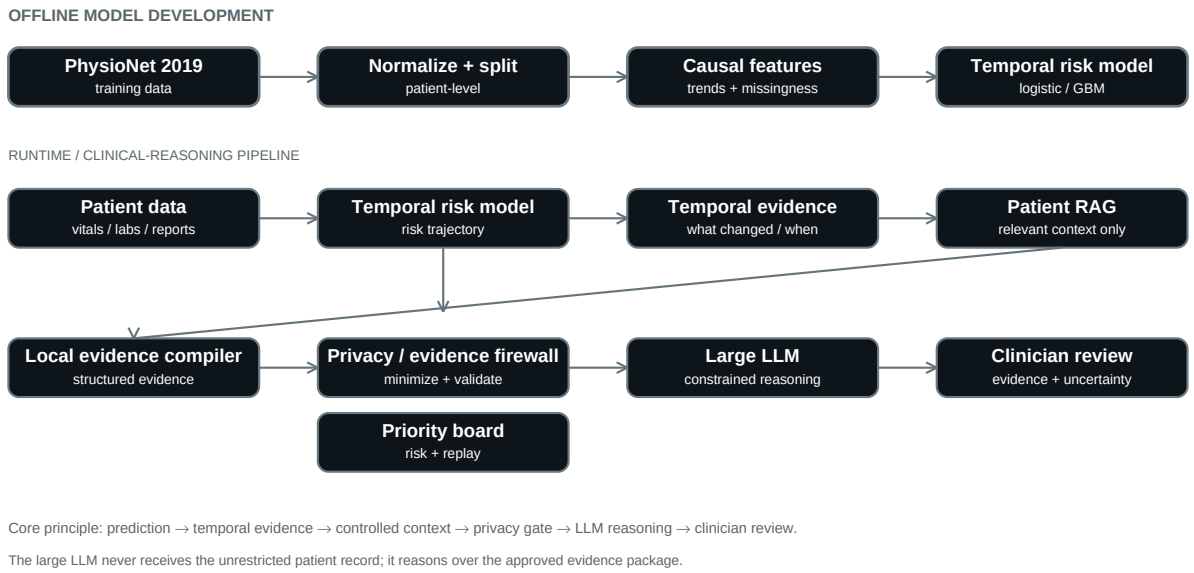
The first version should favor an interpretable model over a complex neural network. The project does not need an LLM or chatbot to qualify as AI; the learned sequential risk estimate is the core AI contribution.

Leakage controls

- Split data by patient, never by individual hourly rows.
- Keep all records from a patient in exactly one of train, validation, or test sets.
- Fit imputation, scaling, feature selection, and calibration on training data only.
- Calculate every rolling feature using only rows available at that prediction time.

- Do not use future labels, future vitals, or full-stay patient averages.
- Tune the alert threshold on validation patients and report final performance on held-out test patients.

System architecture / flow diagram



Core principle: prediction → temporal evidence → controlled context → privacy gate → LLM reasoning → clinician review.

The large LLM never receives the unrestricted patient record; it reasons over the approved evidence package.

Suggested API surface

```
GET  /                        Dashboard page
GET  /api/patients            Patient priority data
GET  /api/patients/{id}       Current patient state
GET  /api/patients/{id}/history Risk and vital history
POST /api/replay/start        Start playback
POST /api/replay/pause        Pause playback
POST /api/replay/step         Advance one simulated hour
POST /api/replay/reset        Reset replay state
WS   /ws/replay               Stream observations and predictions
```

Feasibility

The prototype is feasible within a hackathon because:

- The primary dataset and feature schema are well defined.
- The ML model can be implemented with standard scikit-learn components.
- The replay engine can use a CSV rather than requiring device integration.
- FastAPI provides REST and WebSocket support with little infrastructure.
- HTML/CSS/JavaScript allows a polished interface without adopting a large frontend framework.
- The first model can be trained offline and loaded by the API.
- The scope is intentionally narrow: sepsis-related early warning rather than general diagnosis.

5. Impact, Scalability & Roadmap

Impact metrics

The prototype should report both model performance and operational usefulness:

- **Lead time:** Time between the first sustained model alert and the retrospective labelled onset.
- **Sensitivity:** Percentage of sepsis-positive patients alerted before onset.
- **False alerts per patient-day:** Alert burden at a selected operating threshold.
- **Sensitivity at fixed false-alert rate:** Compare the ML model with the threshold baseline.
- **Patient-level AUROC and AUPRC:** Report performance on held-out patients.
- **Calibration:** Assess whether risk scores behave consistently as probability-like outputs.
- **Coverage of explanations:** Percentage of alerts with a readable evidence summary.
- **Replay latency:** Time from receiving a row to displaying the updated risk.

The primary demonstration should show one patient whose risk rises before the labelled onset while a stable patient remains low priority.

Scalability beyond the prototype

The architecture can evolve in stages:

1. Replace CSV replay with an authenticated EHR, HL7/FHIR, message-queue, or device gateway.
2. Replace in-memory replay state with durable storage and audit logging.
3. Add user authentication, role-based access, and organization-level data isolation.
4. Add model monitoring, drift detection, data-quality checks, and alert acknowledgement workflows.
5. Validate on external hospitals and different patient populations.
6. Add prospective silent-mode evaluation before any clinical workflow integration.
7. Support additional deterioration outcomes only after separate label and validation work.

MVP roadmap

Phase 1 — Project setup and data contract

- Confirm target definition and six-hour prediction horizon.
- Download and inspect the PhysioNet dataset.
- Normalize the six vital-sign fields and missingness indicators.
- Define the synthetic replay schema.

Phase 2 — Feature and model pipeline

- Build causal rolling features.
- Implement the threshold baseline.
- Train logistic regression with a patient-level split.
- Save the model and calculate validation metrics.

Phase 3 — Backend replay service

- Implement FastAPI endpoints.
- Load synthetic demo cases.

- Add step, pause, play, speed, and reset controls.
- Broadcast patient updates over WebSocket.

Phase 4 — Frontend dashboard

- Build the clinical command-center layout.
- Add patient priority cards and vital-sign cards.
- Add risk and vital-sign charts.
- Add alert markers, evidence text, and missing-data indicators.
- Add an always-visible simulation disclaimer.

Phase 5 — Evidence and final demo

- Replay one worsening and one stable patient.
- Compare model alerts with the threshold baseline.
- Calculate lead time and patient-level results.
- Prepare limitations and validation notes.
- Package the code, model artifact, synthetic data, and PDF proposal.

Suggested hackathon schedule

Time	Deliverable
Hours 0–4	Scope, data contract, repository setup, synthetic cases
Hours 4–10	Data normalization and causal feature pipeline
Hours 10–16	Baseline, logistic regression, patient-level evaluation
Hours 16–24	FastAPI endpoints and replay WebSocket
Hours 24–34	Frontend dashboard, charts, and visual polish
Hours 34–40	Connect model, tune replay, write explanations
Hours 40–48	Test, capture evidence, rehearse demo, finalize documentation

6. References & Appendix

Research and datasets

1. **PhysioNet/CinC Challenge 2019: Early Prediction of Sepsis from Clinical Data**

Dataset and challenge information: https://physionet.org/content/challenge-2019/1.0.0/

2. **PhysioNet**

Open research resources for physiological and clinical data: https://physionet.org/

3. **Reyna, M. A. et al.** Early Prediction of Sepsis from Clinical Data: The PhysioNet/Computing in Cardiology Challenge 2019. The challenge publication and citation information are available through the PhysioNet Challenge 2019 resource page.

Technical documentation

4. **FastAPI documentation**

https://fastapi.tiangolo.com/

5. **FastAPI WebSockets**

https://fastapi.tiangolo.com/advanced/websockets/

6. **scikit-learn documentation**

https://scikit-learn.org/stable/

7. **Apache ECharts documentation**

https://echarts.apache.org/en/index.html

Appendix A — Example synthetic patient trajectory

```
Patient: P001
Mode: Synthetic demonstration
Resolution: One observation per simulated hour

08:00  HR 82   RR 17  SpO2 98  BP 122/76  Temp 36.8  Risk 0.08
09:00  HR 85   RR 18  SpO2 97  BP 119/74  Temp 37.0  Risk 0.12
10:00  HR 89   RR 20  SpO2 96  BP 114/71  Temp 37.3  Risk 0.19
11:00  HR 94   RR 22  SpO2 95  BP 108/68  Temp 37.7  Risk 0.31
12:00  HR 99   RR 24  SpO2 94  BP 102/64  Temp --    Risk 0.44
13:00  HR 104  RR 26  SpO2 92  BP 96/60   Temp 38.4  Risk 0.67 ALERT
14:00  HR 108  RR 29  SpO2 91  BP 91/57   Temp 38.7  Risk 0.81 LABELLED ONSET
```

Appendix B — Demonstration language

Use the following wording in the dashboard and presentation:

> *“VitalWatch is a retrospective hourly-data replay demonstrating sequential risk updating. Alerts prioritize clinician assessment; they do not diagnose sepsis or recommend treatment.”*

> *“The explanation lists model-associated evidence, not proven clinical causes.”*

> *“A datathon model supports a research demonstration. Clinical deployment requires prospective, external, and workflow-specific validation.”*

Appendix C — Scope boundaries

The MVP deliberately does not claim:

- Second-by-second waveform monitoring.
- Continuous sensor integration.
- Clinical diagnosis.
- Treatment recommendations.
- Autonomous escalation.
- Generalization to all hospitals or patient groups.
- Prospective clinical benefit.

Appendix D — New AI Idea & Research Framing

The original temporal early-warning system remains the core predictive component. The new idea adds a controlled reasoning layer on top of it.

Updated one-line project statement

A system that combines temporal patient data with privacy-preserving LLM reasoning to detect deterioration and explain what might be happening.

What is new

- Temporal risk model: **Is the patient deteriorating?**
- Temporal evidence engine: **What changed, when, how fast, and how persistent?**
- Patient RAG: **What relevant historical context is needed?**
- Local evidence compiler: retrieved chunks → minimal structured evidence.
- Privacy / evidence firewall: validation, identifier minimization, leakage and prompt-injection checks.
- Large LLM: **What plausible explanations should a clinician consider, given this evidence?**

Correct framing

The LLM is not the primary detector and is not an autonomous diagnostic system. It is the **reasoning and explanation layer constrained by the temporal model**.

Research question

Does constraining an LLM with temporal physiological evidence and selectively retrieved patient context produce more reliable, explainable and privacy-conscious clinical reasoning than asking an LLM to reason directly from patient data?

This extends the original temporal deterioration idea rather than replacing it.

Appendix E — Privacy-Preserving Patient RAG

The RAG layer is not an unrestricted patient-memory system. Retrieval is conditioned by the deterioration event or reasoning task identified by the temporal system.

Stage	Role
Private patient data	Vitals, labs, reports, medications and prior history remain inside the controlled patient-data boundary.
Temporal evidence engine	Defines the current deterioration pattern and reasoning context.
Relevance-bounded Patient RAG	Retrieves only information relevant to the current event/task, with provenance.
Local evidence compiler	Converts raw retrieved chunks into strict structured evidence rather than forwarding raw text.
Privacy / evidence firewall	Identifier minimization, schema validation, leakage checks, prompt-injection checks and provenance.
Large LLM	Receives only the approved evidence package and performs constrained reasoning.
Clinician review	Shows differential possibilities, evidence, uncertainty and traceability.

Example

If the temporal system detects respiratory deterioration, retrieval may request recent respiratory reports, relevant medications and prior respiratory history. Unrelated history should not be retrieved simply because it exists.

Structured evidence passed to the LLM

Field	Example
Temporal event	Respiratory deterioration pattern
Observed change	SpO2 falling while respiratory rate rises
Persistence / baseline	Pattern persists and increasingly deviates from recent baseline
Relevant context	Only retrieved information necessary for interpretation
Contradictions / missing data	Evidence against an interpretation and unavailable context
Provenance	Where the item came from and why it was retrieved

Privacy evaluation

- Direct identifier extraction and indirect leakage tests.
- Prompt-injection tests on retrieved clinical notes.
- Minimum-disclosure comparison.
- Do not claim perfect anonymity; use privacy-preserving / privacy-conscious language.

Appendix F — Constrained LLM Reasoning & Updated Roadmap

The large LLM receives a structured evidence package instead of the raw patient chart. Its output is constrained so reasoning remains traceable and uncertainty is explicit.

LLM output contract

Output	Requirement
Differential possibilities	Plausible explanations, not confirmed diagnoses.
Supporting evidence	Major claims linked to supplied time-stamped evidence.
Evidence against	Contradictory or weakening evidence.
Temporal reasoning	Direction, persistence and timing.
Missing information	What would reduce uncertainty.
Uncertainty	Speculation clearly separated from evidence.
Evidence trace	Reviewer can inspect source evidence.

Experimental comparison

Snapshot LLM	Temporal LLM	Proposed system
Current measurements → LLM	Current + history → LLM	Temporal model → evidence → selective RAG → local compiler → firewall → LLM

Updated roadmap

- **Data/model:** retain PhysioNet 2019, patient-level splits, causal features and temporal ML.
- **Backend:** retain FastAPI replay/prediction service and WebSocket stream.
- **Frontend:** retain historical replay and priority board; add temporal-evidence view.
- **Reasoning:** add event-conditioned Patient RAG, local evidence compilation and provenance.
- **Privacy:** add evidence firewall and leakage / prompt-injection evaluation.
- **LLM:** add constrained differential reasoning with evidence, uncertainty and traceability.

Final safety framing

Alerts prioritize review. The LLM generates plausible differential insights from supplied evidence; it does not independently diagnose, prescribe treatment or escalate care. This remains a research prototype.

Architecture principle

Prediction → evidence → controlled context → privacy gate → reasoning → clinician review.

Appendix G - Future Development

SMARTPHONE-BASED ACTIVE HEALTH MONITORING | NOT PART OF VITALWATCH v1.0

Purpose. A later version of VitalWatch may accept selected, patient-initiated measurements captured with a smartphone. These observations would complement medical reports, not replace them. The current MVP remains the historical/synthetic early-warning replay and its temporal evidence, Patient RAG, privacy/evidence firewall and constrained LLM pipeline.

Planned smartphone capabilities

1. Camera-based resting heart-rate estimate

A guided fingertip-on-camera-and-flash session uses photoplethysmography (PPG) to estimate pulse rate. The app records the estimate, timestamp, device information and signal-quality result. Raw video should be processed on-device and discarded by default.

Output: Resting heart-rate estimate (BPM) plus quality metadata. This is not an ECG, blood-pressure or arrhythmia measurement.

2. Guided mobility assessment

A later app flow may guide a five-times sit-to-stand assessment. On-device video pose estimation or motion sensors can estimate repetition count and completion time, subject to a safe setup and appropriate screening.

Output: Completion time, repetition count and assessment-quality metadata. It is a functional measure, not a diagnosis.

How the future data would join the existing system

Guided phone session Patient starts a test	**On-device processing** Signal / pose + quality checks	**Structured observation** Value, unit, time, source, quality	**Existing VitalWatch pipeline** Temporal model → evidence → RAG → privacy gate → LLM

Integration principle: Each phone-derived result becomes a timestamped, source-labelled observation in the same general data contract used for other patient measurements. The existing temporal model and reasoning pipeline remain the core; a new input adapter supplies the additional observations.

Data, validation and privacy requirements

Data contract	Validation before any clinical use	Privacy safeguards
Observation type; value and unit; timestamp; source/device; signal or test quality; consent status; evidence ID. Preserve whether a value is measured, extracted, patient-reported or simulated.	Compare camera pulse estimates with an appropriate reference such as ECG; compare mobility counts/timing with trained-observer or reference measurements. Test repeatability, device variation, poor-signal rejection and usability in the intended population.	Obtain explicit consent; process video/audio locally where possible; avoid retaining raw media by default; minimize identifiers; encrypt stored data; apply role-based access and audit logging. Never send raw media to an external LLM by default.

Possible later expansion

After the smartphone workflow is independently tested, VitalWatch may add opt-in integrations with compatible wearables and external devices (for example, validated blood-pressure monitors or pulse oximeters). These sources should use the same timestamped observation structure and must retain device/source provenance. Device integration is a future extension, not a dependency of the current project.

Current MVP boundary: Smartphone sensing, wearables and external-device integration are explicitly excluded from the present implementation and primary experiments. VitalWatch v1.0 continues to use the defined clinical dataset, synthetic chronological replay, temporal risk model, temporal evidence, Patient RAG, privacy/evidence firewall and constrained LLM reasoning. Any future sensor-derived model use requires separate technical and clinical validation.

VitalWatch — Detailed Technical Revision

Updated architecture and implementation addendum. The original proposal is retained; this revision replaces the previous SLM-dependent design with a deterministic, no-SLM privacy pipeline.

NO-SLM DESIGN	MVP DECISION
No internal small language model is required for RAG retrieval or privacy protection.	Use rules, regex, NER/PII detection, structured schemas and deterministic validation before any external LLM call.

What is being retained

The original pitch's early-warning foundation remains: chronological patient observations, causal trend features, missingness indicators, patient-level evaluation, a replayable interface and a clinician-review safety boundary. The expanded system adds durable patient records, Redis context caching, patient-scoped RAG, a privacy/evidence firewall, constrained LLM explanations and Agent Orchestrator verification.

Updated project statement

VitalWatch is a longitudinal healthcare intelligence research prototype that organizes patient observations over time, forecasts selected clinical measurements or deterioration risk, retrieves relevant historical evidence, and produces evidence-constrained explanations for clinician review.

The temporal model remains the predictive authority. RAG retrieves relevant historical context. A deterministic evidence-processing and privacy pipeline validates, minimizes and screens that context. The external LLM is the reasoning and explanation layer constrained by temporal model outputs and approved evidence. An orchestrator checks the generated response before it reaches the dashboard.

Scope clarification

The PhysioNet/CinC Challenge 2019 sepsis early-warning task remains a benchmark for the first deterioration-risk experiment. The broader longitudinal report/RAG workflow is an architectural extension and must be validated separately. Do not imply that one dataset validates every VitalWatch capability.

1. Updated System Architecture

The system is divided into a trusted patient-data plane, a temporal prediction plane and a controlled explanation plane. The prediction path and evidence path remain independently testable and meet only at the context builder.

Architecture flow

INPUTS: patient observations · uploaded reports · synthetic data
INGESTION: OCR / text & table extraction → structured observation extraction → schema, unit, date and duplicate validation
STORAGE: PostgreSQL permanent source of truth ↔ Redis patient-context cache
PREDICTION PATH: validated observations → temporal feature engineering → ML/DL models → forecasts, trends and risk metadata
EVIDENCE PATH: private patient-scoped vector index → RAG retrieval → rules / regex / NER structuring → evidence validation → privacy gate → approved minimized evidence
REASONING PATH: predictions + approved evidence + relevant cached context → external LLM → Agent Orchestrator verification → dashboard → clinician review

No SLM is present in this architecture. The privacy gate is a deterministic service with explicit allow/block decisions and auditable rules.

Core component responsibilities

Layer	Responsibility
Ingestion	Convert reports and incoming observations into timestamped, source-linked structured records.
PostgreSQL	Permanent source of truth for patient records, observations, provenance and audit metadata.
Redis	Cache recent patient context, summaries and recent outputs with TTL/versioning; never replace durable storage.
Temporal models	Estimate future values or deterioration risk using only observations available at prediction time.
Patient RAG	Retrieve only relevant evidence from the authorized patient's private index.
Privacy gate	Detect identifiers, minimize data, validate allowed fields and block unsafe external transmission.
External LLM	Explain supplied model outputs and approved evidence; it does not calculate the prediction.
Orchestrator	Verify evidence support, numeric consistency, chronology, schema and safety before display.

2. Data Ingestion, Storage & Patient Context

2.1 Unified ingestion pipeline

- **Sources:** historical benchmark data, synthetic patient trajectories, manually entered observations and uploaded PDF/image reports.
- **Document processing:** OCR plus text and table extraction. Preserve page, table, row and source-document references.
- **Structured extraction:** parameter, value, unit, timestamp, source, extraction method and confidence.
- **Validation:** schema checks, unit normalization, plausible-date checks, duplicate detection, conflict detection and missingness flags.
- **Persistence:** write validated observations to the durable database; retain raw source files under access control.

Example observation contract

```
{
  "patient_id": "P001",
  "parameter": "hemoglobin",
  "value": 11.2,
  "unit": "g/dL",
  "timestamp": "2026-10-15T09:30:00",
  "source_reference": "REPORT_001",
  "extraction_method": "table_parser",
  "confidence": 0.96,
  "is_synthetic": false
}
```

Confidence scores are signals for review, not proof that an extracted value is correct. Preserve the original text/value and source reference so errors can be traced and corrected.

2.2 PostgreSQL and Redis responsibilities

PostgreSQL — durable source of truth	Redis — short-lived context cache
Patient profiles, normalized observations, report metadata, source provenance, prediction records and audit events.	Recent observations, recent trend summaries, recent prediction references and bounded conversation context.
Versioned updates, access control, backup and retention policies.	Patient-scoped keys, TTL, freshness/version checks and invalidation after record changes.

Redis must never silently override newer database records. If a cache entry is stale, missing or inconsistent, fetch from the database and recompute the relevant context. Avoid placing unnecessary raw medical text or identifiers in cache keys and logs.

2.3 Example cache key namespace

```
patient:{id}:recent_observations
patient:{id}:trend_summary
patient:{id}:recent_predictions
patient:{id}:conversation_context
```

3. Temporal Analysis & Prediction

The temporal model is the predictive authority. It receives validated, timestamped observations and produces a structured prediction record. The LLM must not invent, modify or independently determine the forecast.

3.1 Prediction tasks

Task	Example target	Candidate methods
Near-term deterioration risk	Probability of a defined event within a fixed future horizon.	Logistic regression, XGBoost, LSTM, GRU, GRU-D.
Numeric forecasting	Future heart rate, glucose, hemoglobin or creatinine at a specified horizon.	Persistence baseline, linear models, XGBoost, LSTM/GRU.
Trend / anomaly detection	Deviation from a patient's recent pattern.	Robust statistical baselines, Isolation Forest, autoencoder.
Multi-parameter trajectory	Joint future movement of selected measurements.	Independent parameter models first; compare shared multi-output models later.

3.2 Temporal features

- Current measurements and patient-specific historical baselines.
- Causal rolling means, minima, maxima, variability and slopes over selected windows.
- Elapsed time since the last observation and irregular sampling intervals.
- Missingness masks and counts; an unmeasured value must never be treated as normal.
- Persistence and direction of change, calculated using past data only.

3.3 Model output contract

```
{
  "prediction_id": "PRED_001",
  "target": "deterioration_risk",
  "risk_score": 0.72,
  "forecast_horizon_hours": 6,
  "generated_at": "2026-11-08T10:00:00",
  "model_version": "risk_model_v1",
  "uncertainty": "moderate",
  "input_observation_ids": ["OBS_101", "OBS_102"]
}
```

Every prediction must include its target definition, horizon, timestamp, model version and uncertainty metadata. Predictions with stale inputs or failed quality checks should be withheld or clearly marked.

4. Research Methodology & Model Evaluation

The research contribution should be demonstrated through controlled comparisons and ablation studies, not by claiming a novel architecture without evidence.

4.1 Experimental questions

- Does explicit elapsed-time information improve forecasting compared with ordinary LSTM or GRU models?
- Do missingness masks and time-since-last-observation features improve performance over conventional imputation?
- Do patient-specific baselines and causal trend features improve over current-value-only models?
- Does a shared multi-output model outperform independent models, or introduce negative transfer?
- Can uncertainty estimation identify forecasts that should be withheld from the explanation layer?
- How much do OCR and extraction errors degrade downstream prediction?

4.2 Comparison matrix

Experiment	Model / condition	Purpose
E0	Persistence / last observation carried forward	Simple forecasting reference.
E1	Linear or logistic regression with causal features	Interpretable baseline.
E2	XGBoost / gradient-boosted trees	Strong tabular comparator.
E3	LSTM	Sequence-learning baseline.
E4	GRU	Lower-complexity recurrent comparator.
E5	GRU-D or mask + elapsed-time model	Missingness-aware sequence modelling.
E6	Time-aware LSTM/GRU with patient baseline features	Test irregular timing and individual baselines.
E7	Attention sequence model or shared multi-output model	Optional extension if data volume supports it.

4.3 Evaluation protocol

- Split by patient, not by hourly row. Keep each patient's full timeline in exactly one split.
- Within each patient, preserve chronology; at time t, features may use only data available at or before t.
- Fit imputation, scaling, feature selection, calibration and thresholds on training/validation data only.
- Tune thresholds on validation patients and report final results on held-out test patients.
- Report subgroup and missingness-stratified performance where sample sizes permit.

Regression: MAE, RMSE, R^2 and error by horizon. Classification: AUROC, AUPRC, sensitivity, specificity, precision, recall, F1, calibration and false alerts per patient-day. Early-warning utility: lead time and sensitivity at a fixed false-alert burden.

5. Patient-specific RAG & Evidence Handling

RAG supplies relevant historical context; it is not the prediction engine and is not itself a privacy mechanism. Retrieval must be scoped to the authenticated patient and the minimum evidence needed for the current task.

5.1 Retrieval workflow

Step	Action	Control
1. Authorization	Authenticate the user and verify access to the requested patient.	Deny access before retrieval if authorization fails.
2. Patient-scoped search	Query only the authorized patient's private vector index.	Enforce patient ID filtering server-side; do not rely on prompt instructions.
3. Task-conditioned retrieval	Retrieve the smallest relevant set of report chunks and observations.	Use relevance thresholds, date filters and source metadata.
4. Evidence structuring	Use deterministic parsing, regex, terminology maps and NER to map facts to a fixed schema.	Unknown or ambiguous fields remain unknown; do not infer missing facts.
5. Evidence validation	Check schema, source IDs, dates, units, values, negation and contradictions.	Quarantine invalid or conflicting evidence for review.
6. Privacy gate	Detect direct identifiers and review quasi-identifiers; minimize data for the task.	Block transmission when policy checks fail.
7. Evidence package	Assemble only approved facts and source references.	Support local-only mode; record transmission metadata.

5.2 No-SLM implementation choices

- **PII detection:** Microsoft Presidio or equivalent NER plus regex for names, phone numbers, email addresses, IDs and other direct identifiers.
- **Structured extraction:** deterministic patterns, clinical terminology dictionaries, table parsers and explicit field mappings.
- **Validation:** Pydantic/JSON schema, unit normalization, date checks, source-ID validation and numerical consistency rules.
- **Minimization:** remove unnecessary identifiers; use relative time when exact dates are not needed; retain clinically relevant values, units and ordering.
- **Fail-closed behavior:** if detection, validation or policy checks fail, do not send the package to the external LLM.

This design is simpler, cheaper and easier to audit for an MVP than adding a local SLM. Its limitation is reduced flexibility on complex narrative, ambiguous references and context-dependent extraction; uncertain cases should be flagged for human review rather than guessed.

6. Privacy Gate & External LLM Boundary

The privacy pipeline is a separate, testable control between private retrieval and external reasoning. Removing names alone does not guarantee anonymity: exact dates, rare diagnoses, unusual combinations and contextual details may still identify a person.

6.1 Example minimized evidence

```
{
  "subject_reference": "SUBJECT_001",
  "observations": [
    {
      "parameter": "haemoglobin",
      "value": 12.8,
      "unit": "g/dL",
      "relative_time": "T1",
      "source_reference": "REPORT_001"
    }
  ]
}
```

The example retains a clinically meaningful value, unit, relative time and evidence reference while omitting direct identifiers. The exact fields sent must depend on the task and applicable policy.

6.2 Privacy controls

- Keep raw reports and the private vector index in the controlled environment; encrypt storage and transport.
- Use patient-scoped authorization, least privilege, audit logging and retention controls.
- Minimize fields before external transmission; do not send full reports or full-chart prompts by default.
- Treat embeddings and retrieved text as sensitive; apply the same access controls to the vector database.
- Scan for direct identifiers and review quasi-identifiers and rare combinations.
- Provide a local-only option that disables external LLM reasoning.
- Log which categories of data were transmitted and the model/prompt version without copying unnecessary raw medical data into logs.

6.3 Privacy evaluation

Test	What to measure
Direct identifier leakage	Names, IDs, phone numbers, addresses and other identifiers present in the outbound package.
Indirect disclosure	Risk from exact dates, rare conditions, unusual combinations or contextual details.
Clinical fact preservation	Whether values, units, negation, temporality and source references remain correct.
Minimization	Whether every transmitted field is necessary for the explanation task.
Fail-safe behavior	Whether ambiguous or failed checks block external transmission.

Privacy should be reported as measured leakage and minimization performance, not as a claim of guaranteed anonymity.

7. External LLM Reasoning & Agent Orchestrator

The external LLM explains supplied evidence and model outputs. It does not replace the temporal model, calculate clinical forecasts or make autonomous diagnoses. The orchestrator is a bounded verification workflow rather than an unrestricted medical-agent debate.

7.1 Explanation flow

Context Builder → External LLM → Evidence Verification → Prediction Verification → Temporal / Contradiction Verification → Safety & Schema Verification → Pass / bounded retry / abstain

7.2 Verification agents

Verifier	What it checks	Preferred method
Evidence verifier	Each material claim maps to supplied evidence IDs or prediction outputs.	Claim-to-source mapping; deterministic reference checks.
Prediction verifier	Values, units, risk score, forecast horizon, timestamps and model version match the prediction service.	Deterministic Python checks.
Temporal / contradiction verifier	No future data, reversed chronology, unsupported causality or contradiction with supplied evidence.	Timestamp rules; NLI/classifier only as a secondary signal.
Safety / schema verifier	Valid output structure, uncertainty language and no authoritative diagnosis or treatment instruction.	Pydantic schema and policy rules.
Revision controller	Structured correction feedback and at most one retry.	Bounded state machine; abstain if still invalid.

7.3 Required response contract

- **Summary:** concise description of the observed temporal pattern.
- **Observed evidence:** measurements, direction of change, persistence and source references.
- **Model output:** prediction target, value/risk, horizon, model version and uncertainty.
- **Possible interpretations:** clearly labelled hypotheses, never confirmed diagnoses.
- **Evidence against / missing information:** relevant contradictions and unavailable measurements.
- **Traceability:** each material claim links to evidence IDs or prediction output IDs.
- **Safety:** clinician review required; no autonomous diagnosis, prescription or escalation.

If verification fails after one revision, return an explicit unable-to-verify or insufficient-evidence response.

8. Backend Services, APIs & Dashboard

8.1 Suggested service boundaries

Service	Responsibility
Patient / Access API	Authentication, patient authorization and role-aware access.
Ingestion API	Report upload, OCR/extraction jobs, validation and durable writes.
Temporal Feature Service	Causal feature calculation and missingness/time-gap handling.
Prediction Service	Model inference, horizon metadata, uncertainty and model versioning.
RAG Service	Patient-scoped vector retrieval with source provenance and relevance filtering.
Evidence & Privacy Service	Structured extraction, PII detection, minimization, policy validation and outbound allow/block.
Explanation Service	Context assembly and external LLM call using approved evidence only.
Orchestrator Service	Evidence, numerical, temporal, contradiction and safety verification.
Dashboard API / WebSocket	Patient timeline, trends, replay state, prediction updates and verified explanations.

8.2 API sketch

```
POST /patients/{id}/reports
POST /patients/{id}/observations
GET  /patients/{id}/timeline
GET  /patients/{id}/trends?parameter=heart_rate
POST /patients/{id}/predictions
POST /patients/{id}/evidence/explain
GET  /patients/{id}/explanations/{explanation_id}
WS   /replay/{session_id}
```

API responses should include timestamps, units, source IDs, model versions, forecast horizon and validation state. Avoid returning raw patient context to clients that do not need it.

8.3 Dashboard modules

- Patient priority board and chronological timeline.
- Vital-sign and lab trend charts with missingness visibility.
- Forecast/risk panel with horizon, uncertainty and model version.
- Evidence panel showing source references and retrieved history.
- Explanation panel separating observed facts, predictions and hypotheses.
- Privacy/transmission status and clinician review controls.
- Synthetic/historical replay labels and evaluation view.

9. Detailed Implementation Plan & Delivery Gates

Build the predictive core and data contracts first. Add retrieval, privacy and explanation only after the temporal pipeline can be tested independently.

Phase	Work items	Exit gate
0. Scope & contracts	Freeze MVP target/horizon; define observation schema, units, patient split strategy, safety language and synthetic scenarios.	Approved PRD, schema and evaluation protocol.
1. Data pipeline	Load benchmark; implement report parsing/OCR interface; normalize dates/units; add provenance and validation.	Golden extraction tests pass; invalid/conflicting values are flagged.
2. Temporal baseline	Build causal rolling features, missingness masks, elapsed-time features, threshold baseline and logistic/XGBoost model.	Patient-level baseline metrics reproducible; no future leakage.
3. Sequence experiments	Train LSTM/GRU and missingness-aware/time-aware variants; compare ablations and uncertainty.	Held-out evaluation and failure analysis documented.
4. Durable store + Redis	Persist patient timeline; add patient-scoped cache, TTL, invalidation and stale-prediction checks.	Cache cannot override source of truth; isolation tests pass.
5. Patient RAG	Chunk reports with source IDs; create patient-scoped vector index; add task-conditioned retrieval and prompt-injection tests.	Retrieval is access-controlled, relevant and traceable.
6. No-SLM privacy gate	Implement PII detection, regex/NER, structured rules, minimization, schema/unit/date validation and fail-closed policy.	Outbound package passes privacy/fidelity tests; unsafe package is blocked.
7. LLM + orchestrator	Add context builder, constrained prompt, external LLM call, deterministic verifiers and bounded retry/abstention.	Claims trace to evidence; numerical and temporal checks pass.
8. Dashboard & demo	Integrate timeline, charts, prediction/evidence panels, replay and review controls.	Stable demo with synthetic and approved benchmark data.
9. Evaluation & report	Run model, extraction, privacy and orchestration experiments; document limitations.	Reproducible results, ablations and honest claims.

The local SLM fine-tuning phase is removed from the MVP plan. It may be reconsidered only as a separately scoped future research experiment if deterministic extraction demonstrably fails on a defined task.

10. Evaluation, Risks & Governance

10.1 End-to-end experiments

Experiment	Pipeline	Purpose
P1	RAG → external LLM (controlled baseline only)	Establish a reference for leakage and explanation quality; use synthetic or approved test data.
P2	RAG → deterministic structuring + privacy gate → external LLM	Evaluate the selected no-SLM MVP architecture.
P3	No external LLM / local-only evidence display	Measure the value of local-only operation and compare user-facing evidence utility.

Compare direct identifier leakage, indirect disclosure risk, clinical fact preservation, numerical/time consistency, source traceability, explanation quality, latency and token usage. Use synthetic and appropriately authorized de-identified data for testing; do not expose real patient data to unapproved services.

10.2 Risks and mitigations

Risk	Mitigation / honest claim
Dataset shift / limited generalization	Patient-level splits, external validation where possible; one benchmark does not generalize to every hospital.
Label limitations	Define the target operationally; retrospective labels are not exact biological onset or diagnosis.
Missing / irregular measurements	Use masks and elapsed time; report performance by missingness; unmeasured is not normal.
OCR / extraction errors	Preserve source evidence, validate units/dates, flag conflicts and allow human correction.
Stale Redis context	TTL, versioning, invalidation, DB fallback and stale-prediction rejection.
Privacy leakage	Local-first storage, minimization, PII checks, quasi-identifier review, transmission controls and testing.
LLM hallucination	Evidence-linked output, deterministic checks, bounded retry, abstention and clinician review.
Automation bias	Visually separate facts, predictions and hypotheses; never present generated interpretation as confirmed diagnosis.

10.3 Governance boundary

VitalWatch is a research prototype for prioritizing clinician review. It does not independently diagnose, prescribe treatment or autonomously escalate care. Clinical use would require prospective evaluation, external validation, governance approval, privacy/security review and workflow-specific testing.

11. Technical Stack & References

Layer	Suggested technology	Purpose
Backend	Python, FastAPI, Uvicorn	REST APIs, replay, orchestration and inference.
Data processing	Pandas, NumPy, Pydantic	Normalization, causal features and schema validation.
ML	scikit-learn, XGBoost, PyTorch	Baselines and sequence-model experiments.
Storage	PostgreSQL + Redis	Durable patient records and short-lived context cache.
Vector retrieval	FAISS / Chroma / pgvector (choose one for MVP)	Private patient-scoped evidence retrieval.
Privacy	Microsoft Presidio + regex / NER + custom rules	PII detection, minimization and outbound checks.
LLM integration	Approved external LLM API; optional local-only mode	Evidence-constrained explanation.
Orchestration	Python state machine or LangGraph	Bounded verification and retry workflow.
Frontend	HTML/CSS/JavaScript + ECharts or Chart.js	Timeline, trends, risk and evidence dashboard.

Technical references

- PhysioNet / Computing in Cardiology Challenge 2019 — Early Prediction of Sepsis from Clinical Data: physionet.org/content/challenge-2019.
- Microsoft Presidio documentation: microsoft.github.io/presidio.
- FastAPI documentation: fastapi.tiangolo.com.
- scikit-learn documentation: scikit-learn.org.
- PyTorch documentation: pytorch.org/docs.
- Redis documentation: redis.io/docs.
- Hugging Face Transformers documentation: huggingface.co/docs/transformers (for optional future research only, not required by MVP).

Final architecture principle

Validated observations → temporal features and predictions → patient-scoped RAG → deterministic evidence structuring and privacy gate → constrained external reasoning → orchestrator verification → clinician review.

No SLM is required in the current VitalWatch architecture. The MVP prioritizes transparent, testable and auditable processing. Any later language-model component must be justified by measured improvement over the deterministic baseline and must pass the same privacy and clinical-fidelity tests.